

AI ASSISTED CODING

Lab Assignment-3.1

2303A51463

B-07

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Gemini

G Generate a Python function that checks whether a given number is a palindrome.

◆ Here's a Python function to check if a number is a palindrome:

The `is_palindrome_number` function has been executed with example values, and the results are:

- `121` is a palindrome (True).
- `123` is not a palindrome (False).
- `1221` is a palindrome (True).
- `-121` is not a palindrome (False), as the negative sign breaks the symmetry.
- `0` is a palindrome (True).

This confirms the function works as expected for these cases.



[2]

```
def is_palindrome_number(number):  
  
    # Convert the number to a string to easily reverse it  
    num_str = str(number)  
    # Compare the string with its reversed version  
    return num_str == num_str[::-1]  
  
# Example usage:  
print(f"Is 121 a palindrome? {is_palindrome_number(121)}")  
print(f"Is 123 a palindrome? {is_palindrome_number(123)}")  
print(f"Is 1221 a palindrome? {is_palindrome_number(1221)}")  
print(f"Is -121 a palindrome? {is_palindrome_number(-121)}")  
print(f"Is 0 a palindrome? {is_palindrome_number(0)}")
```

▼

```
... Is 121 a palindrome? True  
Is 123 a palindrome? False  
Is 1221 a palindrome? True  
Is -121 a palindrome? False  
Is 0 a palindrome? True
```

[4]

✓ Os

```
print(is_palindrome_number(121))  
print(is_palindrome_number(123))  
print(is_palindrome_number(1221))  
print(is_palindrome_number(10))
```

▼

```
True  
False  
True  
False
```

Edge case testing:

```
[6]
✓ 0s
▶ print(is_palindrome_number(0))
  print(is_palindrome_number(-121))

... True
  False
```

Question 2: One-Shot Prompting (Factorial Calculation)

Gemini

⌵ ⋮ >

G One-Shot Prompt Used Generate a Python function to compute the factorial of a number. Example: Input: 5 → Output: 120

◆ Based on the one-shot prompt you provided, here is the Python function an AI would likely generate to compute the factorial of a given number, including the example usage:

The `calculate_factorial_one_shot` function has been executed. Here are the results:

- **Factorial of 5:** 120 (Correct, as $5! = 5 * 4 * 3 * 2 * 1 = 120$)
- **Factorial of 0:** 1 (Correct, as $0!$ is defined as 1)
- **Factorial of 7:** 5040 (Correct, as $7! = 7 * 6 * 5 * 4 * 3 * 2 * 1 = 5040$)
- **Factorial of -3:** Factorial is not defined for negative numbers. (Correct, as the function explicitly handles negative inputs by returning this message)

This demonstrates that the function correctly computes factorials for non-negative integers and handles negative input as specified.

👍 🗨

```
+ Code + Text
▶ def calculate_factorial_one_shot(n):
    |
    if not isinstance(n, int):
        return "Input must be an integer."
    if n < 0:
        return "Factorial is not defined for negative numbers."
    elif n == 0:
        return 1
    else:
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result

    # Example usage based on the prompt:
    print(f"Factorial of 5: {calculate_factorial_one_shot(5)}")

    # Additional examples to test the function:
    print(f"Factorial of 0: {calculate_factorial_one_shot(0)}")
    print(f"Factorial of 7: {calculate_factorial_one_shot(7)}")
    print(f"Factorial of -3: {calculate_factorial_one_shot(-3)}")

...
Factorial of 5: 120
Factorial of 0: 1
Factorial of 7: 5040
Factorial of -3: Factorial is not defined for negative numbers.
```

Question 3: Few-Shot Prompting (Armstrong Number Check)

6 Few-Shot Prompt Used Generate a Python function to check whether a number is an Armstrong number.

Examples: Input: 153 → Output: Armstrong Number Input: 370 → Output: Armstrong Number Input: 123 → Output: Not an Armstrong Number

◆ Based on the few-shot prompt you provided, here is a Python function to check for Armstrong numbers, along with explanations and examples:

The `is_armstrong_number` function has been executed with the provided examples. Here are the results:

- Is 153 an Armstrong number? `True` (Correct, $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$)
- Is 370 an Armstrong number? `True` (Correct, $3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370$)
- Is 123 an Armstrong number? `False` (Correct, $1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36 \neq 123$)
- Is 371 an Armstrong number? `True` (Correct, $3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371$)
- Is 9474 an Armstrong number? `True` (Correct, $9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474$)
- Is 10 an Armstrong number? `False` (Correct, $1^2 + 0^2 = 1 \neq 10$)
- Is -153 an Armstrong number? `Input must be a non-negative integer.` (Correct, as the function handles invalid negative input).

These results confirm that the function correctly identifies Armstrong numbers and handles edge cases like 0 and negative inputs as intended.

[8]

```
def is_armstrong_number(number):
    if not isinstance(number, int) or number < 0:
        return "Input must be a non-negative integer."
    if number == 0:
        return True # 0 is generally considered an Armstrong number

    # Convert number to string to find its length (number of digits)
    num_str = str(number)
    num_digits = len(num_str)
    sum_of_powers = 0
    for digit_char in num_str:
        digit = int(digit_char)
        sum_of_powers += digit ** num_digits

    # Compare the sum with the original number
    return sum_of_powers == number

print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}")
print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}")
print(f"Is 123 an Armstrong number? {is_armstrong_number(123)}")

# Additional examples to test the function:
print(f"Is 371 an Armstrong number? {is_armstrong_number(371)}") # 3^3 + 7^3 + 1^3 = 371
print(f"Is 9474 an Armstrong number? {is_armstrong_number(9474)}") # 9^4 + 4^4 + 7^4 + 4^4 = 9474
print(f"Is 10 an Armstrong number? {is_armstrong_number(10)}")
print(f"Is -153 an Armstrong number? {is_armstrong_number(-153)}")
```

✓

```
... Is 153 an Armstrong number? True
Is 370 an Armstrong number? True
Is 123 an Armstrong number? False
Is 371 an Armstrong number? True
Is 9474 an Armstrong number? True
Is 10 an Armstrong number? False
Is -153 an Armstrong number? Input must be a non-negative integer.
```

Boundary value testing:

```
[10] ✓ 0s  print(is_armstrong_number(0))  
print(is_armstrong_number(1))  
print(is_armstrong_number(9474))  
... True  
True  
True
```

Question 4: Context-Managed Promptings

Gemini



Context-Managed Prompt Used Generate an optimized Python function to classify a number as Prime, Composite, or Neither. Constraints: • Accept only integers • Handle negative numbers, 0, and 1 correctly • Use an efficient algorithm (check divisors up to $\sqrt{n}$) • Return clear string outputs: "Prime", "Composite", "Neither" • Avoid unnecessary computations

◆ Here is the Python function to classify a number as Prime, Composite, or Neither, based on your context-managed prompt and constraints, including examples.

The `classify_number` function has been executed with a comprehensive set of test cases, demonstrating its adherence to the specified constraints. Here's a breakdown of the results:

- **Prime Numbers:** 2, 3, 17, 97, 127 are correctly classified as 'Prime'.
- **Composite Numbers:** 4, 25, 100, 121 are correctly classified as 'Composite'.
- **'Neither' Numbers:** 0, 1, and -5 are correctly classified as 'Neither', as per mathematical definitions.
- **Input Validation:** Non-integer inputs like 5.5 and 'abc' are correctly handled by returning the error message `Input must be an integer.`

These results confirm that the function correctly classifies numbers and effectively handles edge cases and invalid input types, fulfilling all the prompt's constraints.



```
[13] import math
✓ Os def classify_number(number):
    if not isinstance(number, int):
        return "Input must be an integer."
    if number <= 1:
        return "Neither"
    if number == 2:
        return "Prime"
    if number % 2 == 0:
        return "Composite"
    for i in range(3, int(math.sqrt(number)) + 1, 2):
        if number % i == 0:
            return "Composite"
    return "Prime"
print(f"Classify 2: {classify_number(2)}") # Expected: Prime
print(f"Classify 3: {classify_number(3)}") # Expected: Prime
print(f"Classify 4: {classify_number(4)}") # Expected: Composite
print(f"Classify 0: {classify_number(0)}") # Expected: Neither
print(f"Classify 1: {classify_number(1)}") # Expected: Neither
print(f"Classify 97: {classify_number(97)}") # Expected: Prime
print(f"Classify 100: {classify_number(100)}") # Expected: Composite
print(f"Classify 121: {classify_number(121)}") # Expected: Composite (11*11)
print(f"Classify 127: {classify_number(127)}") # Expected: Prime
print(f"Classify 5.5: {classify_number(5.5)}") # Expected: Input must be an integer.
print(f"Classify 'abc': {classify_number('abc')}") # Expected: Input must be an integer.
```

... Classify 2: Prime
Classify 3: Prime
Classify 4: Composite
Classify 0: Neither
Classify 1: Neither
Classify 97: Prime
Classify 100: Composite
Classify 121: Composite
Classify 127: Prime
Classify 5.5: Input must be an integer.
Classify 'abc': Input must be an integer.

Question 5: Zero-Shot Prompting (Perfect Number Check)

Gemini



G Generate a Python function that checks whether a given number is a perfect number.

◆ Here's a Python function to check whether a given number is a perfect number, along with its definition and examples:

The `is_perfect_number` function has been executed. Here are the results of the example usage:

- `6`, `28`, and `496` are correctly identified as perfect numbers (`True`).
- `10`, `1`, `0`, `-28`, and `7` are correctly identified as not perfect numbers (`False`).

This demonstrates that the function accurately determines whether a given integer is a perfect number and handles various inputs, including positive, zero, negative, and prime numbers, according to the definition.



[18]

```

def is_perfect_number(number):
    if not isinstance(number, int) or number <= 0:
        return False
    if number == 1:
        return False
    sum_divisors = 1 # Start with 1, as 1 is a proper divisor for all numbers > 1
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            sum_divisors += i
            if i * i != number:
                sum_divisors += number // i
    return sum_divisors == number

print(f"Is 6 a perfect number? {is_perfect_number(6)}") # Expected: True
print(f"Is 28 a perfect number? {is_perfect_number(28)}") # Expected: True
print(f"Is 496 a perfect number? {is_perfect_number(496)}") # Expected: True
print(f"Is 10 a perfect number? {is_perfect_number(10)}") # Expected: False
print(f"Is 1 a perfect number? {is_perfect_number(1)}") # Expected: False
print(f"Is 0 a perfect number? {is_perfect_number(0)}") # Expected: False
print(f"Is -28 a perfect number? {is_perfect_number(-28)}") # Expected: False
print(f"Is 7 a perfect number? {is_perfect_number(7)}") # Expected: False (prime number)

```

▼

```

... Is 6 a perfect number? True
    Is 28 a perfect number? True
    Is 496 a perfect number? True
    Is 10 a perfect number? False
    Is 1 a perfect number? False
    Is 0 a perfect number? False
    Is -28 a perfect number? False
    Is 7 a perfect number? False

```

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Gemini

G Few-Shot Prompt Used Generate a Python program that classifies a number as Even or Odd with input validation. Examples: Input: 8 → Output: Even Input: 15 → Output: Odd Input: 0 → Output: Even

◆ Based on your few-shot prompt, here's a Python function to classify a number as Even or Odd, including input validation and examples:

The `classify_even_odd` function has been executed with various inputs. Here's a summary of the results:

- **Even Numbers:** 8, 0, and -4 are correctly classified as 'Even'.
- **Odd Numbers:** 15, 7, and -9 are correctly classified as 'Odd'.
- **Invalid Inputs:** Non-integer inputs like 3.5 and 'hello' are correctly handled by returning 'Invalid Input'.

These results confirm that the function accurately classifies numbers as even or odd and effectively handles different types of inputs, including validation.

[19]

```
def classify_even_odd(number):  
    if not isinstance(number, int):  
        return "Invalid Input"  
    elif number % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"  
print(f"Classify 8: {classify_even_odd(8)}")  
print(f"Classify 15: {classify_even_odd(15)}")  
print(f"Classify 0: {classify_even_odd(0)}")  
  
# Additional examples to test validation and other cases:  
print(f"Classify 7: {classify_even_odd(7)}")  
print(f"Classify -4: {classify_even_odd(-4)}")  
print(f"Classify -9: {classify_even_odd(-9)}")  
print(f"Classify 3.5: {classify_even_odd(3.5)}")  
print(f"Classify 'hello': {classify_even_odd('hello')}")
```

▼

```
... Classify 8: Even  
Classify 15: Odd  
Classify 0: Even  
Classify 7: Odd  
Classify -4: Even  
Classify -9: Odd  
Classify 3.5: Invalid Input  
Classify 'hello': Invalid Input
```